

chapter-2 RMI

Q. Write down the code to create RMI client-server program to calculate simple interest when principal, time and rate are passed from client.

Sol
i) SIImplement.java SIInterface.java:

```
public interface SIInterface extends Remote {  
    public double Interest(double p, double t, double r)  
        throws RemoteException;  
}
```

ii) SIInterfaceImpl.java:

```
public class SIInterfaceImpl extends UnicastRemoteObject  
implements SIInterface {  
    protected SIInterfaceImpl() throws RemoteException {  
        super();  
    }  
  
    public double Interest(double p, double t, double r)  
        throws RemoteException {  
        return ((p*t*r)/100);  
    }  
}
```

iii> SIClient.java:

```
public class SIClient {  
    public static void main (String [] args) {  
        try {  
            SIInterface si = (SIInter
```

iii> SIServer.java:

```
public class SIServer {  
    public static void main (String [] args) {  
        try {  
            SIInterface obj = new SIInterfaceImpl ();  
            LocateRegistry.createRegistry (1099);  
            System.out.println ("Server created successfully.");  
            Naming.rebind ("localhost / hello", obj);  
            System.out.println ("Server ready to serve...");  
        }  
        catch (Exception e) {  
            System.out.println (e);  
        }  
    }  
}
```

iv) SIClient.java:

```
public class SIClient {  
    public static void main (String [] args) {  
        try {  
            BIIInterface si = (BIIInterface) Naming.lookup  
                ("localhost/hello");  
            System.out.println ("Server Says:");  
            double i = si.Interest (12000, 8, 5);  
            System.out.println ("Simple Interest: " + i);  
        }  
        catch (Exception e) {  
            System.out.println ("e");  
        }  
    }  
}
```

② Write a Client/Server RMI program to calculate profit loss. The cp and sp are passed by the client program.

sof code:

i) PLInterface.java:

```
public interface PLInterface extends Remote {  
    public double PLCalculate (double cp, double sp)  
    throws RemoteException;  
}
```

ii) PLInterfaceImpl.java:

```
public class PLInterfaceImpl extends UnicastRemoteObject  
implements PLInterface {  
    protected PLInterfaceImpl() throws RemoteException {  
        super();  
    }  
}
```

```
    public double PLCalculate (double cp, double sp)  
    throws RemoteException {  
        return (sp - cp);  
    }  
}
```

iii) PLServer.java :

```
public class PLServer {  
    public static void main (String [] args) {  
        try {  
            PLInterface PL = new PLInterfaceImpl();  
            LocateRegistry.createRegistry (1099);  
            System.out.println ("Server created successfully...");  
            Naming.rebind ("localhost/hello", obj);  
            System.out.println ("Server ready to serve...");  
        }  
        catch (Exception e) {  
            System.out.println (e);  
        }  
    }  
}
```

iv> PLClient.java:

```
public class PLClient {  
    public static void main (String [] args) {  
        try {  
            PLInterface pl = new PL (PLInterface) Naming.  
lookup ("localhost/hello");  
            System.out.println ("Server Says");  
  
            double PL = pl.PLCalculate (1000, 120.0);  
  
            if ( PL > 0 ) {  
                System.out.println ("Profit");  
            } else {  
                System.out.println ("Loss");  
            }  
        }  
        catch (Exception e) {  
            System.out.println ("e");  
        }  
    }  
}
```

Pre-Board , 2024-Spring

1.b> Write a client/server RMI program to add two number on server and call from client and display sum.

sof Code:

i> AddInterface.java:

```
public interface AddInterface extends Remote {  
    public double Calculate(double x, double y) throws  
    RemoteException;  
}
```

ii> AddInterfaceImpl.java:

```
public class AddInterfaceImpl extends UnicastRemoteObject  
implements AddInterface {  
    protected AddInterfaceImpl() throws RemoteException {  
        super(0);  
    }  
}
```

```
    public double Calculate(double x, double y) throws  
    RemoteException {  
        return (x+y);  
    }  
}
```

iii> AddServer.java;

```
public class AddServer {  
    public static void main (String [] args) {  
        try {  
            AddInterface obj = new AddInterfaceImpl();  
            LocateRegistry.createRegistry (1099);  
            System.out.println ("Server created successfully..");  
            Naming.rebind ("localhost/hello", obj);  
            System.out.println ("Server ready to serve...");  
        }  
        catch (Exception e) {  
            System.out.println (e);  
        }  
    }  
}
```

iv) AddClient.java:

```
public class AddClient {  
    public static void main (String [] args) {  
        try {  
            AddInterface s = (AddInterface) Naming.  
                lookup ("localhost/hello");  
            System.out.println ("Server Says: ");  
  
            double s1 = s.Calculate (3.5, 4.5);  
  
            System.out.println ("Sum : ", +s1);  
        }  
        catch (Exception e) {  
            System.out.println (e);  
        }  
    }  
}
```